CYBERSECURITY PROJECT REPORT

AUTOMATED WEB SCANNER

A Python-based tool for auditing websites for missing or weak HTTP security headers and CORS misconfigurations.

Name: Nisha R

Department: Cyber Security

Institution: Sona College of Technology

Academic Year: 2026–2027

Project topic — Automated Web Scanner

ABSTRACT

Web applications depend on HTTP response headers to enforce browser-side security. Missing or misconfigured headers, and overly permissive Cross-Origin Resource Sharing (CORS) policies, expose users to clickjacking, cross-site scripting, protocol downgrade and cross-origin data theft. This project presents a Python command-line tool that automatically audits one or more websites for these weaknesses, assigns severity ratings and a security grade, and produces console and JSON reports with remediation advice. The tool is intended for defensive education and authorised testing only.

1. INTRODUCTION AND PROBLEM STATEMENT

Manual header review is slow and error-prone, especially across many sites. Security misconfiguration remains in the OWASP Top 10. The goal is to build a lightweight, passive scanner that identifies common header and CORS issues quickly and explains how to fix them.

2. OBJECTIVES

• Detect missing or weak HTTP security headers.

• Detect CORS misconfigurations using crafted Origin headers.

• Rate findings by severity and compute an overall grade.

• Support multiple targets and export results as JSON.

• Keep the tool safe: passive, non-exploitative and permission-based.

3. TOOLS AND TECHNOLOGIES

Component	Purpose
Python 3	Core language
requests	HTTP communication
colorama	Colored terminal output
argparse / json	CLI parsing and report export

4. METHODOLOGY AND DESIGN

The scanner follows a four-stage pipeline: (1) input normalisation of target URLs; (2) a baseline GET request to collect response headers; (3) analysis modules for security headers and CORS; (4) scoring and reporting.

4.1 Security Header Checks

Header	Severity	Protects against
Strict-Transport-Security	High	SSL stripping, downgrade attacks
Content-Security-Policy	High	XSS, data injection
X-Frame-Options	Medium	Clickjacking
X-Content-Type-Options	Medium	MIME sniffing

Referrer-Policy	Low	Referrer information leakage
Permissions-Policy	Low	Unwanted browser feature access

Additional checks: HSTS max-age below six months, CSP containing unsafe-inline or unsafe-eval, headers that disclose technology (Server, X-Powered-By) and sites served over plain HTTP.

4.2 CORS Checks

The tool resends the request with three Origin values: an arbitrary attacker domain, the null origin, and a suffix-bypass origin (target host + .evil-attacker.com). The Access-Control-Allow-Origin and Access-Control-Allow-Credentials responses are analysed:

• Wildcard (*) with credentials = High

• Reflected arbitrary origin with credentials = High

• Reflected origin without credentials = Medium

• Wildcard only = Low (acceptable for public resources)

4.3 Scoring

Starting at 100, each High finding deducts 20 points, Medium 10 and Low 4. Scores map to grades: A (90+), B (75+), C (55+), D (35+), F (below 35).

5. IMPLEMENTATION

The code is a single module (web_scanner.py) organised into functions: normalize(), fetch(), check_security_headers(), check_cors(), scan(), grade() and print_report(). A command-line interface built with argparse accepts URLs, a file of targets, a timeout and a JSON output path. Network errors are handled with try/except so one failing target does not stop a batch scan.

Example usage:

```
python web_scanner.py https://example.com
python web_scanner.py -f urls.txt --json results.json
```

6. TESTING AND RESULTS

The tool should be tested only on authorised targets such as your own deployments or intentionally vulnerable labs (e.g. OWASP Juice Shop, DVWA) running locally. Replace the table below with your own scan results.

Target	Grade	Key findings
http://localhost:3000 (Juice Shop)	[fill]	[e.g. missing CSP, HSTS]
https://example.com	[fill]	[e.g. missing headers]
[your own site]	[fill]	[fill]

[Insert screenshot of scanner output here]

7. ETHICAL AND LEGAL CONSIDERATIONS

The scanner performs only standard GET requests and does not attempt exploitation or brute force. A warning banner reminds users to scan only systems they own or have written permission

to test. Unauthorised scanning may violate computer misuse laws. Results are advisory and must be verified.

8. LIMITATIONS

• Scans only the supplied URL and does not crawl.

• CORS preflight (OPTIONS) behaviour is not evaluated.

• Header presence is checked, but full policy correctness (e.g. complete CSP analysis) is limited.

• Results can differ by endpoint, so a single page may not represent the whole site.

9. FUTURE ENHANCEMENTS

• Cookie flag checks (Secure, HttpOnly, SameSite).

• Crawling to scan multiple pages and endpoints.

• Preflight request testing and HTML/PDF report generation.

• Asynchronous scanning for speed; TLS/certificate checks.

10. CONCLUSION

The Automated Web Scanner demonstrates how common web security misconfigurations can be detected automatically with a small Python tool. It improves speed and consistency over manual review and gives developers actionable remediation guidance, reinforcing secure-by-default practices.

11. REFERENCES

• OWASP Secure Headers Project - owasp.org

• OWASP Top 10: Security Misconfiguration

• MDN Web Docs: HTTP Headers and CORS

• Python requests documentation